

Plan 9 from Bell Labs

Rob Pike, Dave Presotto, Sean Dorward, Bob Flandrena, Ken Thompson, Howard Trickey, Phil Winterbottom

June 4, 2026

Plan 9 from Bell Labs
Rob Pike
Dave Presotto
Sean Dorward
Bob Flandrena
Ken Thompson
Howard Trickey
Phil Winterbottom
Bell Laboratories
Murray Hill, New Jersey 07974
USA

贝尔实验室的 Plan 9
Rob Pike
Dave Presotto
Sean Dorward
Bob Flandrena
Ken Thompson
Howard Trickey
Phil Winterbottom
贝尔实验室
新泽西州默里山 07974
美国

Motivation

By the mid 1980's, the trend in computing was away from large centralized time-shared computers towards networks of smaller, personal machines, typically UNIX 'workstations'. People had grown weary of overloaded, bureaucratic timesharing machines and were eager to move to small, self-maintained systems, even if that meant a net loss in computing power. As microcomputers became faster, even that loss was recovered, and this style of computing remains popular today.

In the rush to personal workstations, though, some of their weaknesses were overlooked. First, the operating system they run, UNIX, is itself an old timesharing system and has had trouble adapting to ideas born after it. Graphics and networking were added to UNIX well into its lifetime and remain poorly integrated and difficult to administer. More important, the early focus on having private machines made it difficult for networks of machines to serve as seamlessly as the old monolithic timesharing systems. Timesharing centralized the management and amortization of costs and resources; personal computing fractured, democratized, and ultimately amplified administrative problems. The choice of an old timesharing operating system to run those personal machines made it difficult to bind things together smoothly.

Plan 9 began in the late 1980's as an attempt to have it both ways: to build a system that was centrally administered and cost-

动机

到20世纪80年代中期, 计算的趋势是从大型集中式分时计算机转向由

然而, 在急于转向个人工作站的过程中, 它们的一些弱点被忽视。UNIX 本身就是一个旧的分时系统, 难以适应后来出现的想法。图形用户界面是 UNIX 生命周期后期才添加的, 至今仍整合不佳且难以管理。更重要的是,

Plan 9 始于20世纪80年代末, 试图兼顾两者: 使用廉价的现代微处理器 [NeHe82] 非常相似。早期的口号是用许多小系统构建一个系统, 而不是用许多小 UNIX 构建一个系统。

effective using cheap modern microcomputers as its computing elements. The idea was to build a time-sharing system out of workstations, but in a novel way. Different computers would handle different tasks: small, cheap machines in people's offices would serve as terminals providing access to large, central, shared resources such as computing servers and file servers. For the central machines, the coming wave of shared-memory multiprocessors seemed obvious candidates. The philosophy is much like that of the Cambridge Distributed System [NeHe82]. The early catch phrase was to build a UNIX out of a lot of little systems, not a system out of a lot of little UNIXes.

The problems with UNIX were too deep to fix, but some of its ideas could be brought along. The best was its use of the file system to coordinate naming of and access to resources, even those, such as devices, not traditionally treated as files. For Plan 9, we adopted this idea by designing a network-level protocol, called 9P, to enable machines to access files on remote systems. Above this, we built a naming system that lets people and their computing agents build customized views of the resources in the network. This is where Plan 9 first began to look different: a Plan 9 user builds a private computing environment and recreates it wherever desired, rather than doing all computing on a private machine. It soon became clear that this model was richer than we had foreseen, and the ideas of per-process name spaces and file-system-like resources were extended throughout the system—to processes, graphics, even the network itself.

By 1989 the system had become solid enough that some of us began using it as our exclusive computing environment. This meant bringing along many of the services and applications we had used on UNIX. We used this opportunity to revisit many issues, not just kernel-resident ones, that we felt UNIX addressed badly. Plan 9 has new compilers, languages, libraries, window systems, and many new applications. Many of the old tools were dropped, while those brought along have been polished or rewritten.

Why be so all-encompassing? The distinction between operating system, library, and application is important to the operating system researcher but uninteresting to the user. What matters is clean functionality. By building a complete new system, we were able to solve problems where we thought they should be solved. For example, there is no real 'tty driver' in the kernel; that is the job

UNIX 的问题太深奥，无法修复，但其一些想法可以借鉴。最好的 Plan 9，我们通过设计一种名为 9P 的网络级协议来采用这个想法，Plan 9 开始看起来与众不同的地方：Plan 9 用户构建一个私人计算——一到进程、图形，甚至网络本身。

到1989年，该系统已经足够稳定，我们中的一些人开始将其作为 UNIX 上使用的许多服务和应用程序。我们利用这个机会重新审视许多 UNIX 处理得不好的问题，不仅是内核级的问题。Plan 9 拥有新的编译器、语言、库、窗口系统和许多新应用程序。许多旧

为什么要如此包罗万象？操作系统、库和应用程序之间的区别对“驱动程序”；这是窗口系统的工作。在现代世界中，多厂商和多架构的 UNIX 主力程序提供更高效的方式是空洞的工程；我们更感兴趣的是 Plan 9 提供了一个运行 POSIX 命令的模拟环境，但它是系统的一个在 Plan 9 环境中开发的。

of the window system. In the modern world, multi-vendor and multi-architecture computing are essential, yet the usual compilers and tools assume the program is being built to run locally; we needed to rethink these issues. Most important, though, the test of a system is the computing environment it provides. Producing a more efficient way to run the old UNIX warhorses is empty engineering; we were more interested in whether the new ideas suggested by the architecture of the underlying system encourage a more effective way of working. Thus, although Plan 9 provides an emulation environment for running POSIX commands, it is a backwater of the system. The vast majority of system software is developed in the 'native' Plan 9 environment.

There are benefits to having an all-new system. First, our laboratory has a history of building experimental peripheral boards. To make it easy to write device drivers, we want a system that is available in source form (no longer guaranteed with UNIX, even in the laboratory in which it was born). Also, we want to redistribute our work, which means the software must be locally produced. For example, we could have used some vendors' C compilers for our system, but even had we overcome the problems with cross-compilation, we would have difficulty redistributing the result.

This paper serves as an overview of the system. It discusses the architecture from the lowest building blocks to the computing environment seen by users. It also serves as an introduction to the rest of the Plan 9 Programmer's Manual, which it accompanies. More detail about topics in this paper can be found elsewhere in the manual.

Design

The view of the system is built upon three principles. First, resources are named and accessed like files in a hierarchical file system. Second, there is a standard protocol, called 9P, for accessing these resources. Third, the disjoint hierarchies provided by different services are joined together into a single private hierarchical file name space. The unusual properties of Plan 9 stem from the consistent, aggressive application of these principles.

A large Plan 9 installation has a number of computers networked together, each providing a particular class of service. Shared multiprocessor servers provide computing cy-

拥有一个全新的系统有很多好处。首先，我们的实验室有构建实现 UNIX 中不再保证，即使是在它诞生的实验室中）。此外，我们希望 C 编译器，但即使我们克服了交叉编译的问题，我们也难以重新分

本文作为系统的概述。它讨论了从最低层构建块到用户所见的计划 9 程序员手册》其余部分的介绍。关于本文主题的更多细节可以在手

设计

系统的视图基于三个原则构建。首先，资源像分层文件系统中的文件。9P 的标准协议用于访问这些资源。第三，不同服务提供的不相交层 9 的独特特性源于这些原则的一致、积极应用。

一个大型 Plan 9 安装有许多联网的计算机，每台提供特定类别的服务（如 ISDN）将这些服务器连接到办公室和家庭中的工作站或 PC，在 Plan 9 术语中称为终端。图 1 显示了这种布置。

cles; other large machines offer file storage. These machines are located in an air-conditioned machine room and are connected by high-performance networks. Lower bandwidth networks such as Ethernet or ISDN connect these servers to office- and home-resident workstations or PCs, called terminals in Plan 9 terminology. Figure 1 shows the arrangement.

The modern style of computing offers each user a dedicated workstation or PC. Plan 9's approach is different. The various machines with screens, keyboards, and mice all provide access to the resources of the network, so they are functionally equivalent, in the manner of the terminals attached to old timesharing systems. When someone uses the system, though, the terminal is temporarily personalized by that user. Instead of customizing the hardware, Plan 9 offers the ability to customize one's view of the system provided by the software. That customization is accomplished by giving local, personal names for the publicly visible resources in the network. Plan 9 provides the mechanism to assemble a personal view of the public space with local names for globally accessible resources. Since the most important resources of the network are files, the model of that view is file-oriented.

The client's local name space provides a way to customize the user's view of the network. The services available in the network all export file hierarchies. Those important to the user are gathered together into a custom name space; those of no immediate interest are ignored. This is a different style of use from the idea of a 'uniform global name space'. In Plan 9, there are known names for services and uniform names for files exported by those services, but the view is entirely local. As an analogy, consider the difference between the phrase 'my house' and the precise address of the speaker's home. The latter may be used by anyone but the former is easier to say and makes sense when spoken. It also changes meaning depending on who says it, yet that does not cause confusion. Similarly, in Plan 9 the name `/dev/cons` always refers to the user's terminal and `/bin/date` the correct version of the `date` command to run, but which files those names represent depends on circumstances such as the architecture of the machine executing `date`. Plan 9, then, has local name spaces that obey globally understood conventions; it is the conventions that guarantee sane behavior in the presence of local names.

现代计算风格为每个用户提供专用的工作站或 PC。Plan 9 的方法不同。各种带有屏幕、键盘和鼠标的机器都提供对网络资源。Plan 9 不是定制硬件，而是提供了定制软件提供的系统视图的能力。这种 Plan 9 提供了一种机制，可以用全局可访问资源的本地名称来组装公共空

客户端的本地命名空间提供了一种定制用户网络视图的方式。网络 Plan 9 中，服务有已知的名称，这些服务导出的文件有统一的名称。Plan 9 中，名称 `/dev/cons` 总是指用户的终端，`/bin/date` 是要运行的 `date` 命令的正确版本，但这些名称代表哪些文件取决于 `date` 的机器架构等情况。因此，Plan 9 有遵守全局理解约定的本地

The 9P protocol is structured as a set of transactions that send a request from a client to a (local or remote) server and return the result. 9P controls file systems, not just files: it includes procedures to resolve file names and traverse the name hierarchy of the file system provided by the server. On the other hand, the client's name space is held by the client system alone, not on or with the server, a distinction from systems such as Sprite [OCDNW88]. Also, file access is at the level of bytes, not blocks, which distinguishes 9P from protocols like NFS and RFS. A paper by Welch compares Sprite, NFS, and Plan 9's network file system structures [Welc94].

This approach was designed with traditional files in mind, but can be extended to many other resources. Plan 9 services that export file hierarchies include I/O devices, backup services, the window system, network interfaces, and many others. One example is the process file system, /proc, which provides a clean way to examine and control running processes. Precursor systems had a similar idea [Kill84], but Plan 9 pushes the file metaphor much further [PPTTW93]. The file system model is well-understood, both by system builders and general users, so services that present file-like interfaces are easy to build, easy to understand, and easy to use. Files come with agreed-upon rules for protection, naming, and access both local and remote, so services built this way are ready-made for a distributed system. (This is a distinction from 'object-oriented' models, where these issues must be faced anew for every class of object.) Examples in the sections that follow illustrate these ideas in action.

The Command-level View

Plan 9 is meant to be used from a machine with a screen running the window system. It has no notion of 'teletype' in the UNIX sense. The keyboard handling of the bare system is rudimentary, but once the window system, 8½ [Pike91], is running, text can be edited with 'cut and paste' operations from a pop-up menu, copied between windows, and so on. 8½ permits editing text from the past, not just on the current input line. The text-editing capabilities of 8½ are strong enough to displace special features such as history in the shell, paging and scrolling, and mail editors. 8½ windows do not support cursor addressing and, except for one terminal emulator to simplify connecting to traditional

9P 协议的结构是一组事务，将请求从客户端发送到（本地或远程）服务器以控制文件系统，而不仅仅是文件：它包括解析文件名和遍历服务器树。这与 Sprite [OCDNW88] 等系统不同。此外，文件访问是在字节级别，而不是块级别，这使 9P 与 NFS 和 RFS 等协议区分开来。Welch 的一篇论文比较了 Sprite、NFS 和 Plan 9 的网络文件系统结构 [Welc94]。

这种方法在设计时考虑了传统文件，但可以扩展到许多其他资源。Plan 9 服务包括 I/O 设备、备份服务、窗口系统、网络接口等等。/proc，它提供了一种检查和控制运行中进程的简洁方式。先驱系统 [Kill84]，但 Plan 9 将文件隐喻推向了更远 [PPTTW93]。文件系统

命令级视图

Plan 9 旨在从运行窗口系统的带屏幕机器上使用。它没有 UNIX 意义上的“电传打字机”概念。裸系统的键盘处理是基本的，8½ [Pike91] 运行，就可以通过弹出菜单的“剪切和粘贴”操作编辑文本，而不仅仅是当前输入行。8½ 的文本编辑能力包括 shell 中的历史记录、分页和滚动以及邮件编辑器等特殊功能。8½ 窗口不支持光标寻址，除了一个用于简化连接到传统系统的终端模拟器。8½ 中没有光标寻址软件。

systems, there is no cursor-addressing software in Plan 9.

Each window is created in a separate name space. Adjustments made to the name space in a window do not affect other windows or programs, making it safe to experiment with local modifications to the name space, for example to substitute files from the dump file system when debugging. Once the debugging is done, the window can be deleted and all trace of the experimental apparatus is gone. Similar arguments apply to the private space each window has for environment variables, notes (analogous to UNIX signals), etc.

Each window is created running an application, such as the shell, with standard input and output connected to the editable text of the window. Each window also has a private bitmap and multiplexed access to the keyboard, mouse, and other graphical resources through files like `/dev/mouse`, `/dev/bitblt`, and `/dev/cons` (analogous to UNIX's `/dev/tty`). These files are provided by 8%, which is implemented as a file server. Unlike X windows, where a new application typically creates a new window to run in, an 8% graphics application usually runs in the window where it starts. It is possible and efficient for an application to create a new window, but that is not the style of the system. Again contrasting to X, in which a remote application makes a network call to the X server to start running, a remote 8% application sees the mouse, bitblt, and cons files for the window as usual in `/dev`; it does not know whether the files are local. It just reads and writes them to control the window; the network connection is already there and multiplexed.

The intended style of use is to run interactive applications such as the window system and text editor on the terminal and to run computation- or file-intensive applications on remote servers. Different windows may be running programs on different machines over different networks, but by making the name space equivalent in all windows, this is transparent: the same commands and resources are available, with the same names, wherever the computation is performed.

The command set of Plan 9 is similar to that of UNIX. The commands fall into several broad classes. Some are new programs for old jobs: programs like `ls`, `cat`, and `who` have familiar names and functions but are new, simpler implementations. `Who`, for example, is a shell script, while `ps` is just 95 lines of C code. Some commands are essentially the same as their UNIX ancestors: `awk`, `troff`, and oth-

每个窗口都在一个单独的命名空间中创建。对窗口中命名空间所 UNIX 信号) 等私有空间。

每个窗口在创建时运行一个应用程序(如 shell), 标准输入和 `/dev/mouse`、`/dev/bitblt` 和 `/dev/cons` (类似于 UNIX 的 `/dev/tty`) 等文件多路访问键盘、鼠标和其他图形资源。这 8% 提供, 8% 实现为文件服务器。与 X 窗口不同(在 X 窗口中, 新应用程序通常创建一个新窗口来运行), 8% 图形应用程序通常在其启动的窗口中运行。应用程序可以高效地创建 X 对比, 在 X 中, 远程应用程序向 X 服务器发出网络调用来开始运行。8% 应用程序像往常一样在 `/dev` 中看到窗口的鼠标、`bitblt` 和 `cons` 文件; 它不知道这些文件是否是本地的。它只是通过读写这

预期的使用方式是在终端上运行交互式应用程序(如窗口系统和

Plan 9 的命令集与 UNIX 相似。命令分为几大类。有些是用于旧 `ls`、`cat` 和 `who` 这样的程序具有熟悉的名称和功能, 但却是新的、更是一个 shell 脚本, 而 `ps` 只有 95 行 C 代码。有些命令本质上与其 UNIX 祖先相同: `awk`、`troff` 和其他命令已转换为 ANSI C 并扩展为处理 Unicode, 但仍然是熟悉的工具。有些是针对 `l`rc、文本编辑器 `sam`、调试器 `acid` 等取代了更知名的 UNIX 工具, 完成类似的工作。最后, 大约一半的命令是新的。

ers have been converted to ANSI C and extended to handle Unicode, but are still the familiar tools. Some are entirely new programs for old niches: the shell `rc`, text editor `sam`, debugger `acid`, and others displace the better-known UNIX tools with similar jobs. Finally, about half the commands are new.

Compatibility was not a requirement for the system. Where the old commands or notation seemed good enough, we kept them. When they didn't, we replaced them.

The File Server

A central file server stores permanent files and presents them to the network as a file hierarchy exported using 9P. The server is a stand-alone system, accessible only over the network, designed to do its one job well. It runs no user processes, only a fixed set of routines compiled into the boot image. Rather than a set of disks or separate file systems, the main hierarchy exported by the server is a single tree, representing files on many disks. That hierarchy is shared by many users over a wide area on a variety of networks. Other file trees exported by the server include special-purpose systems such as temporary storage and, as explained below, a backup service.

The file server has three levels of storage. The central server in our installation has about 100 megabytes of memory buffers, 27 gigabytes of magnetic disks, and 350 gigabytes of bulk storage in a write-once-read-many (WORM) jukebox. The disk is a cache for the WORM and the memory is a cache for the disk; each is much faster, and sees about an order of magnitude more traffic, than the level it caches. The addressable data in the file system can be larger than the size of the magnetic disks, because they are only a cache; our main file server has about 40 gigabytes of active storage.

The most unusual feature of the file server comes from its use of a WORM device for stable storage. Every morning at 5 o'clock, a dump of the file system occurs automatically. The file system is frozen and all blocks modified since the last dump are queued to be written to the WORM. Once the blocks are queued, service is restored and the read-only root of the dumped file system appears in a hierarchy of all dumps ever taken, named by its date. For example, the dump from December 1, 1992, would appear at `/n/dump/1992/12/01`. The dump files are stored as a read-only snapshot and cannot be modified, but they may be ac-

兼容性不是系统的要求。在旧命令或符号看起来足够好的地方，

文件服务器

中央文件服务器存储永久文件，并将它们作为使用 9P 导出的文件层次结构呈现给网络。该服务器是一个独立系统，只能通

文件服务器有三个存储级别。我们安装中的中央服务器有大约 100 兆字节的内存缓冲区、27 千兆字节的磁盘和 350 千兆字节的一次性写入多次读取 (WORM) 自动换盘机大容量存储。在 WORM 的缓存，内存是磁盘的缓存；每一层都比它缓存的层快得多，40 千兆字节的活动存储。

文件服务器最不寻常的特性来自于它使用 WORM 设备进行稳定存储。每天早上 5 点，文件系统会自动进行一次转储到 WORM。一旦块排队完毕，服务就会恢复，转储的文件系统的只读根位于 `/n/dump/1992/12/01`。转储文件作为只读快照存储，不能修改，但可以重新写入 WORM。这使得每日转储高效且可以快速执行。

cessed just like any other file system, allowing users to browse through previous versions of files. The dump mechanism is incremental: only blocks that have changed since the last dump are written to the WORM. This makes the daily dumps efficient and allows them to be performed quickly.

The file server also provides a temporary file system, `/tmp`, which is stored in memory and is thus very fast. Files in `/tmp` are automatically deleted when they are no longer needed (when all references to them are closed). This temporary file system is useful for storing intermediate results during computations or for communication between processes.

Processes

In Plan 9, the process model is built around the concept of lightweight processes and shared resources. Processes in Plan 9 are designed to be efficient and flexible, allowing for easy creation and management of concurrent tasks. The basic process creation mechanism in Plan 9 is through the `rfork` system call. Unlike Unix's `fork`, which creates a complete copy of the parent process, `rfork` allows fine-grained control over which resources are shared between parent and child. This includes the address space, file descriptors, and name space. Processes can share memory, file descriptors, and other resources, enabling efficient communication and resource sharing.

The `/proc` file system provides a clean interface for examining and controlling running processes. Each process has a directory under `/proc` containing files that represent various aspects of the process, such as its memory map, file descriptors, and current state. This allows processes to be inspected and manipulated using standard file system operations. Plan 9 also introduces the concept of namespaces for processes. Each process can have its own private view of the file system namespace, which can be modified without affecting other processes. This allows for flexible and secure resource management, as processes can be given access to only the resources they need.

The Window System

Plan 9's window system, `8½`, is designed as a file server that exports a hierarchical file system interface for graphical resources. Unlike traditional window systems like X, which

文件服务器还提供一个临时文件系统 `/tmp`，它存储在内存中，其中的文件在不再需要时（当所有对它们的引用都关闭时）会自动删除。

进程

在 Plan 9 中，进程模型围绕轻量级进程和共享资源的概念构建。Plan 9 中的进程设计高效且灵活，允许轻松创建和管理并发任务。Plan 9 中基本的进程创建机制是通过 `rfork` 系统调用实现的。与 Unix 的 `fork`（创建父进程的完整副本）不同，`rfork` 允许对父子进程之间共享哪些资源进行细粒度控制，包括地址空间、

`/proc` 文件系统提供了一个简洁的接口来检查和控制运行中的进程。`/proc` 下都有一个目录，包含代表进程各个方面的文件，如其内存映射。Plan 9 还引入了进程的命名空间概念。每个进程可以拥有自己的文件系统。

窗口系统

Plan 9 的窗口系统 `8½` 被设计为一个文件服务器，它导出一个用于图形用户界面的文件接口。与 X 等传统窗口系统（使用带有显式网络协议的客户端-服务器模型）不同，`8½` 无缝集成到 Plan 9 以文件为中心的架构中。其中的每个窗口都表示为 `/dev` 目录中的一个文件。程序通过读写这些

use a client-server model with explicit network protocols, 8½ integrates seamlessly into Plan 9's file-centric architecture. Every window in 8½ is represented as a file in the /dev directory. Programs interact with windows by reading from and writing to these files. The window system handles all the low-level graphics operations, allowing applications to focus on their core functionality without needing to understand the details of the display hardware.

8½ provides a text-based interface, with each window acting as an editable text buffer. This approach simplifies the interaction model and allows for powerful text manipulation capabilities. Users can cut, copy, and paste text between windows, and the system supports Unicode for internationalization. The window system also manages input devices like the keyboard and mouse, exposing them as files in /dev. Applications can read input events from these files, making it easy to handle user interactions in a uniform way.

The Network

Networking in Plan 9 is deeply integrated into the system's file-centric design. All network services are accessed through the file system, using the 9P protocol as the underlying communication mechanism. This approach provides a uniform interface for accessing both local and remote resources. The network stack in Plan 9 is implemented as a user-level file server, rather than being part of the kernel. This allows for greater flexibility and easier development of network protocols. The ip file server handles TCP/IP networking, while other servers provide additional network services.

Plan 9 supports a variety of network protocols, including TCP/IP, and provides tools for network configuration and management. The ndb (network database) system is used to manage network configuration information, replacing traditional Unix configuration files like /etc/hosts and /etc/resolv.conf. One of the key features of Plan 9's network design is its support for distributed computing. Using the 9P protocol, processes on different machines can seamlessly access resources as if they were local. This enables transparent distributed computing, where the network becomes invisible to users and applications.

8½ 提供基于文本的接口，每个窗口作为可编辑的文本缓冲区。这使用 Unicode 以实现国际化。窗口系统还管理键盘和鼠标等输入设备，将 /dev 中的文件公开。应用程序可以从这些文件读取输入事件，从而

网络

Plan 9 中的网络功能深度集成到系统以文件为中心的设计中。所有 9P 协议作为底层通信机制。这种方法为访问本地和远程资源提供了 Plan 9 中的网络栈作为用户级文件服务器实现，而不是内核的一部分。这由文件服务器处理 TCP/IP 网络，而其他服务器提供额外的网络服务。

Plan 9 支持多种网络协议，包括 TCP/IP，并提供网络配置和管理。Unix 配置文件如 /etc/hosts 和 /etc/resolv.conf。Plan 9 网络设计的一个关键特性是其对分布式计算的支持。使用 9P 协议，不同机器上的进程可以无缝访问资源，就像它们是本地资源。

Discussion

Plan 9 represents a radical rethinking of operating system design, building on the Unix philosophy but extending it in new directions. The system's file-centric approach provides a unified and consistent interface for all resources, simplifying both system design and application development. One of the key insights of Plan 9 is the recognition that many of Unix's limitations stem from its origins as a time-sharing system. By reimagining the operating system for the distributed computing era, Plan 9 addresses many of these limitations and provides a more modern approach to system design.

The use of the 9P protocol and per-process name spaces enables seamless distributed computing, making the network transparent to users and applications. This approach has influenced subsequent systems, including Linux's namespace support and container technologies like Docker. While Plan 9 never achieved widespread adoption, its ideas continue to shape the field of operating systems. The system's emphasis on simplicity, consistency, and distributed computing provides valuable lessons for system designers and developers.

Security

Plan 9 takes a different approach to security than most operating systems. Instead of using a traditional Unix-style permission system with user and group IDs, Plan 9 uses a capability-based security model. In this model, access to resources is controlled by capabilities, which are unforgeable tokens that grant specific permissions. The 9P protocol includes support for authentication and authorization. When a client connects to a file server, it must authenticate itself using a secure mechanism. Once authenticated, the client receives capabilities that determine what operations it can perform on the server's resources. This approach provides fine-grained control over access to resources and avoids many of the security vulnerabilities associated with traditional Unix permissions.

Conclusion

Plan 9 from Bell Labs represents a bold rethinking of what an operating system should be. By extending the Unix philosophy of "everything is a file" to its logical conclu-

讨论

Plan 9 代表了对操作系统设计的彻底重新思考，它基于 Unix 理念但向新方向扩展。系统以文件为中心的方法为所有资源提供统一且一致的接口。Plan 9 的关键见解之一是认识到 Unix 的许多局限性源于其作为分时系统的设计。Plan 9 解决了许多这些局限性，并提供了更现代的系统设计方法。

9P 协议和进程级命名空间的使用实现了无缝的分布式计算，使网络对用户和应用程序透明。Linux 的命名空间支持和 Docker 等容器技术。尽管 Plan 9 从未被广泛采用，但其思想继续影响操作系统领域。系统对

安全性

Plan 9 在安全性方面采取了与大多数操作系统不同的方法。Plan 9 不使用传统的 Unix 风格的用户和组 ID 权限系统，而是使用基于能力的模型。9P 协议包括对身份验证和授权的支持。当客户端连接到文件服务器时，系统会验证其身份并授予相应的能力。这避免了与 Unix 权限相关的安全漏洞。

结论

贝尔实验室的 Plan 9 代表了对操作系统应该是什么的大胆重新思考。通过扩展 Unix 的“一切皆文件”理念到其逻辑结论，Plan 9 为所有系统资源创建了统一且一致的接口。进程级命名空间和 9P 协议的使用实现了无缝的分布式计算，使网络对用户和应用程序

sion, Plan 9 creates a unified and consistent interface for all system resources. The use of per-process name spaces and the 9P protocol enables seamless distributed computing, making the network transparent to users and applications. Although Plan 9 never achieved the widespread adoption of Unix or Linux, its ideas have had a profound influence on subsequent operating systems and distributed systems research. Concepts like the `/proc` file system, UTF-8 encoding, and capability-based security have been adopted by many modern systems. Plan 9 remains an important milestone in the history of operating systems, demonstrating what is possible when fundamental assumptions are challenged and reimaged.

Plan 9 从未像 Unix 或 Linux 那样被广泛采用，但其思想对后来的操作系统产生了深远影响。`/proc` 文件系统、UTF-8 编码和基于能力的安全性等概念已被许多现代操作系统采用。Plan 9 仍然是操作系统历史上的一个重要里程碑，展示了当基本假设受到挑战时，操作系统可以如何被重新想象。